

you think will be required in your driver. So, in our case, we find out the minor number of the device from the *inode*, and then compare it with the array of devices to find out the index of the device which has called the open function. We store the array index in that *private_data*. When we open both our devices, we will have two separate *struct* files; so we put the index into *private_data* to know later which *struct* file is for which device.

Let's now look at the write callback function.

```
static ssize_t mydevice_write(struct file *filp, const char
*buff, size_t len, loff_t *off)
{
    char temp;
    int retval = len;
    int devicepin;

    if(copy_from_user(&temp, buff, 1))    //only take the
first byte
        return -EFAULT;
    if(temp<48 || temp>49)                //ascii code of 0 and 1
        return -EINVAL;

    temp -=48;
    devicepin = *(int *) (filp->private_data);
    spin_lock_irq(&pprt_lock);
    switch(devicepin) {
    case 0: //STROBE is the first bit
        control = parport_read_control(pprt->port);
        control &= ~(0x01); //reset the bit
        control |= !temp; // change the bit, 1 because STROBE
is activeLow
        parport_write_control(pprt->port, control);
        break;

    case 1: // second element of array is D3
        data = parport_read_data(pprt->port);
        data &= ~(1<<3);
        data |= (temp<<3);
        parport_write_data(pprt->port, data);
        break;

    default:
        retval = -EINVAL;
        break;
    }

    spin_unlock_irq(&pprt_lock);
    return retval;
}
```

In the write call, we just accept values '0' and '1', get the array index from the *private_data* and based on the switch-case, we control the particular port pin. *parport_read_*

control() and *parport_write_control()* are the functions to read and write to the parallel port control pins. Similarly, *parport_read_data()* and *parport_write_data()* are the functions to read and write to the parallel port data pins. Just like the user space code, *bit0* is to be changed to control the Strobe pin.

In the write call, there are chances of one more problem occurring. Nowadays, there are multiple processors in a system, so what will happen if two different processes from two different processors try to control our devices at the same time? The devices will be left in an inconsistent state. To prevent that, we use *spinlock*. Before modifying the port pins, we use the lock so that any other process cannot enter the same code block that this process is executing. After the pins are modified, we release the lock. To use the lock, we need to define it first:

```
static DEFINE_SPINLOCK(pprt_lock);
```

We have not yet defined our close callback function. Whenever any device is closed, we just free the memory that was allocated in the open call to store the index number.

```
static int mydevice_release(struct inode *inode, struct file
*file)
{
    if(file->private_data) {
        kfree(file->private_data);
        file->private_data = NULL;
    }
    return 0;
}
```

When we *insmod* this driver, if all goes well, we will have two devices: */dev/mydev1* and */dev/mydev2*. To turn *dev1* on or off, we can simply echo 1 or 0 to the device.

To turn on the device, type:

```
echo 1 > /dev/mydev1
```

To turn off the device, type:

```
echo 0 > /dev/mydev1
```

Alternately, you can write a user space program that opens this device and turns it on or off according to your requirement.

You can get the full user space code discussed earlier along with this driver from <https://github.com/sudipm-mukherjee/osfy>. 

By: Sudip Mukherjee

The author is the maintainer of the Silicon Motion SM712 and SM750 framebuffer driver in the mainline kernel. He works as an embedded Linux developer at Vector India Pvt Ltd, a renowned embedded training institute. He can be reached at sudip@vectorindia.org